

Snickr Collaboration System

Part 1 Database Design Report

CS 6083 Database Systems Project
Shuhan Cai sc12227
Yang Zhang yz12043

April 26, 2026

1 Introduction

This report documents the relational database design for *snickr*, a Slack-like collaboration system. The system stores users, workspaces, workspace administrators, invitations, channels, channel memberships, and messages. The database is implemented for PostgreSQL and is intended to be used by a web application in Project 2. Application users do not receive database accounts; the application connects to PostgreSQL and applies access-control checks using the logged-in web session.

The design goal is to keep the schema normalized, enforce core integrity rules inside the DBMS, and leave policy decisions that depend on the logged-in browser session to application code. Every major action has a timestamp so the later web application can sort events, display history, and audit invitations.

2 Assumptions and Design Decisions

- User passwords are represented by `password_hash`; the database never stores plaintext passwords.
- A workspace creator is inserted as an administrator in `workspace_members`. Additional administrators are represented by the same table with role `admin`.
- Workspace membership and channel membership are separate. A user must be in both to read a channel's messages.
- Public, private, and direct channels use one `channels` table. Direct channels are constrained by application logic to exactly two members. This avoids adding a mostly duplicate table for direct messages.
- Invitations are stored separately from accepted memberships so the system can report old pending invitations.
- Content access is not implemented with database users, permissions, or views. Queries join through membership tables and the application supplies the logged-in user's id.
- PostgreSQL `ILIKE` is used for case-insensitive keyword search. The optional `pg_trgm` extension can be enabled later to support a GIN index for efficient substring search.

3 ER Diagram

Figure 1 shows the ER design. Many-to-many relationships such as workspace membership and channel membership are represented as associative entities because they also store attributes such as role and join time.

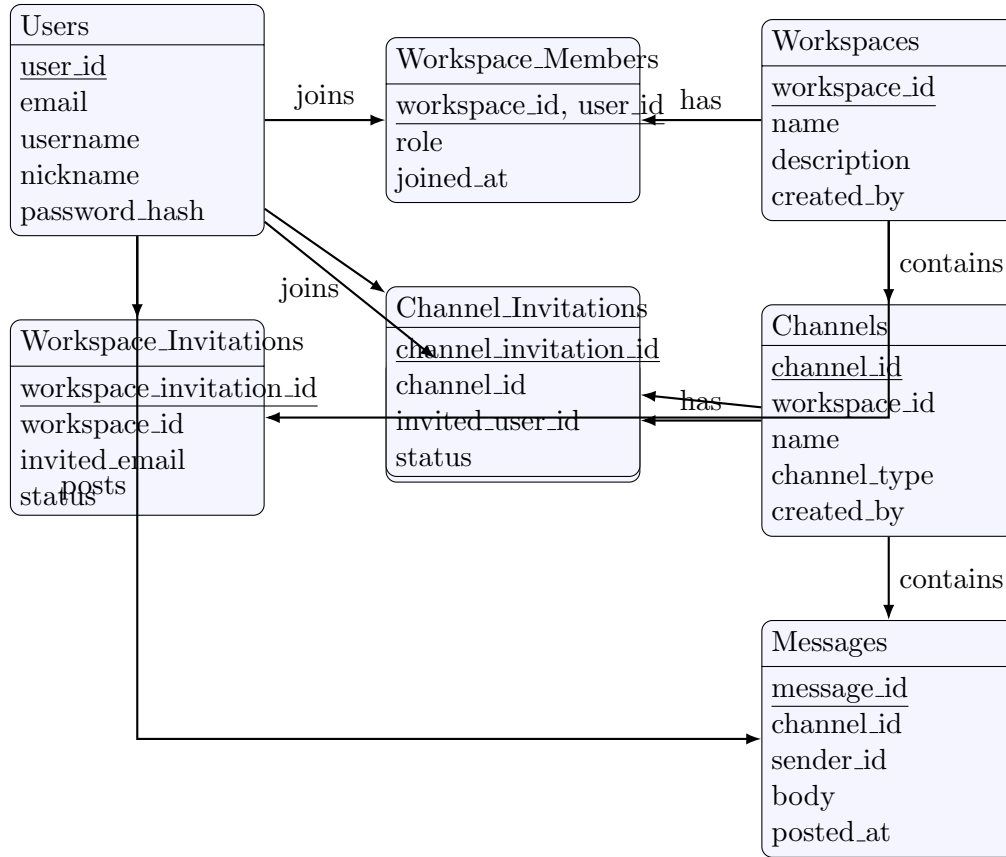


Figure 1: ER diagram for the snickr database.

4 Relational Schema

Table	Attributes	Main constraints
users	user_id email username nickname password_hash created_at	Primary key user_id; unique email; unique username; email must contain @.
workspaces	workspace_id name description created_by created_at	Primary key workspace_id; unique name; created_by references users.

Table	Attributes	Main constraints
workspace_ members	workspace_id user_id role joined_at	Composite primary key (workspace_id, user_id); foreign keys to workspaces and users; role is admin or member.
workspace_ invitations	workspace_invitation_id workspace_id invited_email invited_user_id invited_by status invited_at responded_at	Primary key workspace_invitation_id; unique (workspace_id, invited_email); status is pending, accepted, declined, or revoked.
channels	channel_id workspace_id name channel_type created_by created_at	Primary key channel_id; unique (workspace_id, name); type is public, private, or direct.
channel_ members	channel_id user_id joined_at	Composite primary key (channel_id, user_id); foreign keys to channels and users.
channel_ invitations	channel_invitation_id channel_id invited_user_id invited_by status invited_at responded_at	Primary key channel_invitation_id; unique (channel_id, invited_user_id); status is pending, accepted, declined, or revoked.
messages	message_id channel_id sender_id body posted_at	Primary key message_id; foreign keys to channels and users; body must not be blank.

5 Keys, Foreign Keys, and Other Constraints

Surrogate identity keys are used for entities that have independent existence: users, workspaces, channels, invitations, and messages. Composite primary keys are used for membership tables because the relationship itself is unique: a user can join the same workspace or channel only once.

Foreign keys enforce ownership and membership references. Deleting a workspace cascades to workspace memberships, invitations, channels, and channel-level records. Deleting a channel cascades to its memberships, invitations, and messages. The project statement does not require deletion features, but the cascade rules keep the database internally consistent if an administrative maintenance operation is later added.

Other constraints enforce unique emails, unique usernames, unique workspace names, unique channel names inside a workspace, controlled status values, and controlled channel types. The **messages** table rejects blank message bodies. Indexes support common operations: chronological channel reading, messages by sender, and pending invitation reports. A commented optional `pg_trgm` index is included for faster keyword search when extension creation is allowed.

6 Required SQL Tasks

The full PostgreSQL query file is included in Appendix C. This section summarizes how each required task is handled.

1. A new user is inserted into `users`; uniqueness constraints reject duplicate email addresses and usernames.
2. A public channel is inserted only if an `EXISTS` subquery proves that the creator is a current member of the workspace. The channel creator is added to `channel_members` in the same transaction.
3. Administrators are listed by joining `workspaces`, `workspace_members`, and `users` where role is `admin`.
4. Old pending invitations are counted for public channels using `INTERVAL '5 days'` and a `NOT EXISTS` anti-join against `channel_members`.
5. Channel messages are selected by channel id and ordered by timestamp and message id.
6. A user's posted messages are selected by `sender_id`.
7. Accessible keyword search joins both membership tables before applying `ILIKE` to message body text.

7 Sample Data and Test Structure

The sample data uses twelve users, four workspaces, twelve starting channels, and forty messages. The structure is large enough for a Project 2 browser demo while still being readable in the appendix. It covers public, private, and direct channels; accepted, pending, declined, and revoked invitations; administrators and normal members; direct-message browsing; and keyword-search access control.

Entity or relationship	Rows
Users	12
Workspaces	4
Workspace memberships	17
Workspace invitations	8
Channels	12
Channel memberships	31
Channel invitations	12
Messages	40

Workspace	Members	Channels
Acme Lab	Alice admin, Bob admin, Cara, Maya, Gina, Hank	#general, #product, #sprint-planning, #hiring private, leadership private, Alice/Bob DM, Cara/Maya DM
Coop Board	Cara admin, Alice, Erin, Ken	#maintenance public
Design Guild	Maya admin, Hank admin, Gina, Jack	#design-general public, #ux-research private
Study Hub	Ivy admin, Jack, Lara	#study-general public, #exam-prep private

The key access-control test is the word “perpendicular.” It appears in #general, #hiring, and #maintenance. Cara belongs to Acme Lab and #general, and she is also a member of Coop Board and #maintenance. Therefore, the accessible-message query returns Bob’s public #general message and Alice’s #maintenance message. It does not return Alice’s private #hiring message, because Cara is not a member of that channel. The larger sample also contains other workspaces and private channels that Cara cannot read, giving the browser interface enough data to demonstrate filtered browsing, search results, direct messages, and invitation workflows. This verifies that workspace membership alone is not enough; channel membership is also required.

8 Testing Results

Appendix D contains a PostgreSQL `psql` test script. Appendix E contains the expected result transcript. The tests cover all seven required queries plus an access-control observation showing that private channel messages are hidden from non-members.

9 Project 2 Readiness

This database design is ready to support the Project 2 browser interface. The application should use prepared statements or stored procedures for all parameterized operations. It should HTML-escape message text and nicknames when rendering them to prevent cross-site scripting. Multi-step operations such as creating a channel and adding its first member should run in transactions, as shown in the required query for task 2. User session state should map a browser session cookie to a single `user_id`; all message browsing and searching queries should include that id in membership joins.

A PostgreSQL Schema

```
DROP TABLE IF EXISTS messages CASCADE;
DROP TABLE IF EXISTS channel_invitations CASCADE;
DROP TABLE IF EXISTS channel_members CASCADE;
DROP TABLE IF EXISTS channels CASCADE;
DROP TABLE IF EXISTS workspace_invitations CASCADE;
DROP TABLE IF EXISTS workspace_members CASCADE;
DROP TABLE IF EXISTS workspaces CASCADE;
DROP TABLE IF EXISTS users CASCADE;

CREATE TABLE users (
```

```

    user_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    username VARCHAR(80) NOT NULL UNIQUE,
    nickname VARCHAR(120) NOT NULL,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT users_email_format_ck CHECK (position('@' in email) > 1)
);

CREATE TABLE workspaces (
    workspace_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(120) NOT NULL UNIQUE,
    description TEXT,
    created_by BIGINT NOT NULL REFERENCES users(user_id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspace_members (
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE,
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    role VARCHAR(20) NOT NULL DEFAULT 'member',
    joined_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (workspace_id, user_id),
    CONSTRAINT workspace_members_role_ck CHECK (role IN ('admin', 'member'))
);

CREATE TABLE workspace_invitations (
    workspace_invitation_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE,
    invited_email VARCHAR(255) NOT NULL,
    invited_user_id BIGINT REFERENCES users(user_id),
    invited_by BIGINT NOT NULL REFERENCES users(user_id),
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    invited_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    responded_at TIMESTAMPTZ,
    UNIQUE (workspace_id, invited_email),
    CONSTRAINT workspace_invitations_status_ck
        CHECK (status IN ('pending', 'accepted', 'declined', 'revoked'))
);

CREATE TABLE channels (
    channel_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE,
    name VARCHAR(120) NOT NULL,
    channel_type VARCHAR(20) NOT NULL,
    created_by BIGINT NOT NULL REFERENCES users(user_id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (workspace_id, name),
    CONSTRAINT channels_type_ck CHECK (channel_type IN ('public', 'private', 'direct'))
);

CREATE TABLE channel_members (
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    joined_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (channel_id, user_id)
);

CREATE TABLE channel_invitations (

```

```

channel_invitation_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
invited_user_id BIGINT NOT NULL REFERENCES users(user_id),
invited_by BIGINT NOT NULL REFERENCES users(user_id),
status VARCHAR(20) NOT NULL DEFAULT 'pending',
invited_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
responded_at TIMESTAMPTZ,
UNIQUE (channel_id, invited_user_id),
CONSTRAINT channel_invitations_status_ck
    CHECK (status IN ('pending', 'accepted', 'declined', 'revoked'))
);

CREATE TABLE messages (
    message_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
    sender_id BIGINT NOT NULL REFERENCES users(user_id),
    body TEXT NOT NULL,
    posted_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT messages_body_not_blank_ck CHECK (length(btrim(body)) > 0)
);

CREATE INDEX idx_messages_channel_time ON messages(channel_id, posted_at, message_id);
CREATE INDEX idx_messages_sender_time ON messages(sender_id, posted_at, message_id);
CREATE INDEX idx_channel_invitations_pending ON channel_invitations(channel_id, status, invited_at)
;

-- Optional performance optimization for substring keyword search:
-- CREATE EXTENSION IF NOT EXISTS pg_trgm;
-- CREATE INDEX idx_messages_body_trgm_like ON messages USING gin (body gin_trgm_ops);

```

B Sample Data

```

INSERT INTO users(user_id, email, username, nickname, password_hash, created_at) VALUES
(1, 'alice@example.com', 'alice', 'Alice A.', 'hash_alice', '2026-04-01 09:00:00-04'),
(2, 'bob@example.com', 'bob', 'Bobby', 'hash_bob', '2026-04-01 09:05:00-04'),
(3, 'cara@example.com', 'cara', 'Cara', 'hash_cara', '2026-04-01 09:10:00-04'),
(4, 'dan@example.com', 'dan', 'Dan', 'hash_dan', '2026-04-01 09:15:00-04'),
(5, 'erin@example.com', 'erin', 'Erin', 'hash_erin', '2026-04-01 09:20:00-04'),
(6, 'maya@example.com', 'maya', 'Maya', 'hash_maya', '2026-04-01 09:25:00-04'),
(7, 'gina@example.com', 'gina', 'Gina', 'hash_gina', '2026-04-01 09:30:00-04'),
(8, 'hank@example.com', 'hank', 'Hank', 'hash_hank', '2026-04-01 09:35:00-04'),
(9, 'ivy@example.com', 'ivy', 'Ivy', 'hash_ivy', '2026-04-01 09:40:00-04'),
(10, 'jack@example.com', 'jack', 'Jack', 'hash_jack', '2026-04-01 09:45:00-04'),
(11, 'ken@example.com', 'ken', 'Ken', 'hash_ken', '2026-04-01 09:50:00-04'),
(12, 'lara@example.com', 'lara', 'Lara', 'hash_lara', '2026-04-01 09:55:00-04');

INSERT INTO workspaces(workspace_id, name, description, created_by, created_at) VALUES
(1, 'Acme Lab', 'Engineering collaboration for Acme Lab.', 1, '2026-04-02 08:00:00-04'),
(2, 'Coop Board', 'Building coop board workspace.', 3, '2026-04-03 08:00:00-04'),
(3, 'Design Guild', 'Product design and research workspace.', 6, '2026-04-04 08:00:00-04'),
(4, 'Study Hub', 'Course planning and exam preparation workspace.', 9, '2026-04-05 08:00:00-04')
);

INSERT INTO workspace_members(workspace_id, user_id, role, joined_at) VALUES
(1, 1, 'admin', '2026-04-02 08:00:00-04'),
(1, 2, 'admin', '2026-04-02 10:00:00-04'),

```

```
(1, 3, 'member', '2026-04-03 12:00:00-04'),
(1, 6, 'member', '2026-04-08 09:00:00-04'),
(1, 7, 'member', '2026-04-09 09:00:00-04'),
(1, 8, 'member', '2026-04-10 09:00:00-04'),
(2, 3, 'admin', '2026-04-03 08:00:00-04'),
(2, 1, 'member', '2026-04-04 12:00:00-04'),
(2, 5, 'member', '2026-04-05 12:00:00-04'),
(2, 11, 'member', '2026-04-06 12:00:00-04'),
(3, 6, 'admin', '2026-04-04 08:00:00-04'),
(3, 7, 'member', '2026-04-04 10:00:00-04'),
(3, 8, 'admin', '2026-04-04 11:00:00-04'),
(3, 10, 'member', '2026-04-05 11:00:00-04'),
(4, 9, 'admin', '2026-04-05 08:00:00-04'),
(4, 10, 'member', '2026-04-05 10:00:00-04'),
(4, 12, 'member', '2026-04-06 10:00:00-04');
```

```
INSERT INTO workspace_invitations(workspace_invitation_id, workspace_id, invited_email,
invited_user_id, invited_by, status, invited_at, responded_at) VALUES
(1, 1, 'dan@example.com', 4, 1, 'pending', '2026-04-15 09:00:00-04', NULL),
(2, 1, 'cara@example.com', 3, 1, 'accepted', '2026-04-02 09:00:00-04', '2026-04-03 12:00:00-04'
),
(3, 2, 'alice@example.com', 1, 3, 'accepted', '2026-04-03 09:00:00-04', '2026-04-04 12:00:00-04'
'),
(4, 1, 'erin@example.com', 5, 2, 'pending', '2026-04-20 09:00:00-04', NULL),
(5, 3, 'dan@example.com', 4, 6, 'declined', '2026-04-07 09:00:00-04', '2026-04-08 16:30:00-04')
,
(6, 4, 'alice@example.com', 1, 9, 'pending', '2026-04-18 14:00:00-04', NULL),
(7, 3, 'jack@example.com', 10, 6, 'accepted', '2026-04-04 09:00:00-04', '2026-04-05 11:00:00-04'
'),
(8, 2, 'maya@example.com', 6, 3, 'revoked', '2026-04-10 09:30:00-04', '2026-04-12 17:00:00-04')
;
```

```
INSERT INTO channels(channel_id, workspace_id, name, channel_type, created_by, created_at) VALUES
(1, 1, 'general', 'public', 1, '2026-04-02 08:15:00-04'),
(2, 1, 'product', 'public', 2, '2026-04-05 10:00:00-04'),
(3, 1, 'hiring', 'private', 1, '2026-04-06 11:00:00-04'),
(4, 1, 'dm-alice-bob', 'direct', 1, '2026-04-07 12:00:00-04'),
(5, 2, 'maintenance', 'public', 3, '2026-04-04 09:00:00-04'),
(6, 1, 'sprint-planning', 'public', 6, '2026-04-08 09:15:00-04'),
(7, 1, 'leadership', 'private', 1, '2026-04-09 10:00:00-04'),
(8, 1, 'dm-cara-maya', 'direct', 3, '2026-04-10 11:00:00-04'),
(9, 3, 'design-general', 'public', 6, '2026-04-04 08:30:00-04'),
(10, 3, 'ux-research', 'private', 8, '2026-04-05 13:00:00-04'),
(11, 4, 'study-general', 'public', 9, '2026-04-05 08:30:00-04'),
(12, 4, 'exam-prep', 'private', 9, '2026-04-06 15:00:00-04');
```

```
INSERT INTO channel_members(channel_id, user_id, joined_at) VALUES
(1, 1, '2026-04-02 08:15:00-04'),
(1, 2, '2026-04-02 10:00:00-04'),
(1, 3, '2026-04-03 12:05:00-04'),
(2, 1, '2026-04-05 10:00:00-04'),
(2, 2, '2026-04-05 10:01:00-04'),
(3, 1, '2026-04-06 11:00:00-04'),
(3, 2, '2026-04-06 11:05:00-04'),
(4, 1, '2026-04-07 12:00:00-04'),
(4, 2, '2026-04-07 12:00:00-04'),
(5, 3, '2026-04-04 09:00:00-04'),
(5, 1, '2026-04-04 12:00:00-04'),
(6, 1, '2026-04-08 09:15:00-04'),
```

```
(6, 2, '2026-04-08 09:16:00-04'),
(6, 3, '2026-04-08 09:17:00-04'),
(6, 6, '2026-04-08 09:18:00-04'),
(6, 7, '2026-04-09 09:10:00-04'),
(7, 1, '2026-04-09 10:00:00-04'),
(7, 2, '2026-04-09 10:05:00-04'),
(8, 3, '2026-04-10 11:00:00-04'),
(8, 6, '2026-04-10 11:00:00-04'),
(9, 6, '2026-04-04 08:30:00-04'),
(9, 7, '2026-04-04 10:05:00-04'),
(9, 8, '2026-04-04 11:05:00-04'),
(9, 10, '2026-04-05 11:05:00-04'),
(10, 6, '2026-04-05 13:00:00-04'),
(10, 8, '2026-04-05 13:05:00-04'),
(11, 9, '2026-04-05 08:30:00-04'),
(11, 10, '2026-04-05 10:05:00-04'),
(11, 12, '2026-04-06 10:05:00-04'),
(12, 9, '2026-04-06 15:00:00-04'),
(12, 12, '2026-04-06 15:05:00-04');
```

```
INSERT INTO channel_invitations(channel_invitation_id, channel_id, invited_user_id, invited_by,
status, invited_at, responded_at) VALUES
(1, 1, 4, 1, 'pending', '2026-04-15 09:00:00-04', NULL),
(2, 2, 3, 2, 'pending', '2026-04-17 09:00:00-04', NULL),
(3, 2, 4, 2, 'pending', '2026-04-24 09:00:00-04', NULL),
(4, 3, 3, 1, 'pending', '2026-04-16 09:00:00-04', NULL),
(5, 5, 1, 3, 'accepted', '2026-04-03 09:00:00-04', '2026-04-04 12:00:00-04'),
(6, 6, 4, 6, 'pending', '2026-04-18 09:00:00-04', NULL),
(7, 6, 5, 6, 'pending', '2026-04-23 09:00:00-04', NULL),
(8, 7, 3, 1, 'pending', '2026-04-19 09:00:00-04', NULL),
(9, 9, 10, 6, 'accepted', '2026-04-04 09:00:00-04', '2026-04-05 11:00:00-04'),
(10, 10, 7, 8, 'declined', '2026-04-06 09:00:00-04', '2026-04-07 10:00:00-04'),
(11, 11, 11, 9, 'pending', '2026-04-20 09:00:00-04', NULL),
(12, 12, 10, 9, 'revoked', '2026-04-07 09:00:00-04', '2026-04-08 12:00:00-04');
```

```
INSERT INTO messages(message_id, channel_id, sender_id, body, posted_at) VALUES
(1, 1, 1, 'Welcome to Acme Lab general.', '2026-04-02 08:20:00-04'),
(2, 1, 2, 'The perpendicular test rig is ready.', '2026-04-03 10:00:00-04'),
(3, 1, 3, 'I can review the onboarding checklist.', '2026-04-03 12:30:00-04'),
(4, 2, 2, 'Product roadmap review at 3pm.', '2026-04-05 10:10:00-04'),
(5, 3, 1, 'Private hiring notes mention perpendicular leadership fit.', '2026-04-06 12:00:00-04'),
(6, 4, 1, 'Bob, can you check the direct channel setup?', '2026-04-07 12:05:00-04'),
(7, 5, 3, 'Maintenance elevator request is open.', '2026-04-04 09:30:00-04'),
(8, 5, 1, 'The stair railing is perpendicular to the landing.', '2026-04-04 12:15:00-04'),
(9, 2, 1, 'Launch checklist moved to the product board.', '2026-04-08 09:20:00-04'),
(10, 2, 1, 'Please review the signup copy before Friday.', '2026-04-08 10:05:00-04'),
(11, 2, 1, 'Roadmap notes now include the browser demo path.', '2026-04-08 11:30:00-04'),
(12, 6, 6, 'Sprint planning starts with the database migration review.', '2026-04-08 09:30:00-04'),
(13, 6, 3, 'I will bring the invitation edge cases.', '2026-04-08 09:45:00-04'),
(14, 6, 7, 'The QA checklist needs one more search scenario.', '2026-04-09 09:40:00-04'),
(15, 7, 1, 'Leadership notes: keep admin controls simple.', '2026-04-09 10:15:00-04'),
(16, 7, 1, 'Only channel members should see this budget thread.', '2026-04-09 10:45:00-04'),
(17, 8, 3, 'Maya, can you help with the onboarding screen?', '2026-04-10 11:10:00-04'),
(18, 8, 6, 'Yes, I will mock up the workspace selector.', '2026-04-10 11:12:00-04'),
(19, 5, 3, 'The elevator vendor confirmed Tuesday morning.', '2026-04-05 09:15:00-04'),
(20, 5, 1, 'I added the maintenance note to the shared log.', '2026-04-05 12:30:00-04'),
(21, 5, 3, 'We should invite Ken to the next board update.', '2026-04-06 09:10:00-04');
```

```

(22, 9, 6, 'Welcome to the design guild workspace.', '2026-04-04 08:45:00-04'),
(23, 9, 7, 'I posted color contrast examples for the mockup.', '2026-04-04 10:20:00-04'),
(24, 9, 8, 'Research notes are linked from the channel topic.', '2026-04-04 11:30:00-04'),
(25, 9, 10, 'I can test the responsive layout tomorrow.', '2026-04-05 11:30:00-04'),
(26, 10, 6, 'Interview script draft is ready for review.', '2026-04-05 13:15:00-04'),
(27, 10, 8, 'Private research findings should stay in this channel.', '2026-04-05 13:45:00-04')
),
(28, 11, 9, 'Study Hub is open for database review.', '2026-04-05 08:45:00-04'),
(29, 11, 10, 'I uploaded the normalization notes.', '2026-04-05 10:30:00-04'),
(30, 11, 12, 'Can we schedule a transaction isolation review?', '2026-04-06 10:30:00-04'),
(31, 12, 9, 'Exam prep channel starts with query optimization.', '2026-04-06 15:20:00-04'),
(32, 12, 12, 'I will summarize indexes and explain plans.', '2026-04-06 15:45:00-04'),
(33, 6, 1, 'The demo account list should include invited users.', '2026-04-10 09:00:00-04'),
(34, 2, 1, 'Search examples should include roadmap and browser.', '2026-04-10 10:00:00-04'),
(35, 9, 6, 'The dashboard should show unread channel counts.', '2026-04-10 11:00:00-04'),
(36, 10, 8, 'User quotes need anonymized labels.', '2026-04-10 13:00:00-04'),
(37, 11, 9, 'Bookmarkable workspace URLs would help during the demo.', '2026-04-10 14:00:00-04')
),
(38, 12, 12, 'Prepared statements are on my study checklist.', '2026-04-10 15:00:00-04'),
(39, 4, 1, 'This direct message verifies one-to-one browsing.', '2026-04-11 12:00:00-04'),
(40, 8, 3, 'Direct messages should appear only to both participants.', '2026-04-11 12:30:00-04')
);

SELECT setval(pg_get_serial_sequence('users', 'user_id'), (SELECT max(user_id) FROM users));
SELECT setval(pg_get_serial_sequence('workspaces', 'workspace_id'), (SELECT max(workspace_id) FROM
workspaces));
SELECT setval(pg_get_serial_sequence('workspace_invitations', 'workspace_invitation_id'), (SELECT
max(workspace_invitation_id) FROM workspace_invitations));
SELECT setval(pg_get_serial_sequence('channels', 'channel_id'), (SELECT max(channel_id) FROM
channels));
SELECT setval(pg_get_serial_sequence('channel_invitations', 'channel_invitation_id'), (SELECT max(
channel_invitation_id) FROM channel_invitations));
SELECT setval(pg_get_serial_sequence('messages', 'message_id'), (SELECT max(message_id) FROM
messages));

```

C Required Queries

```

-- Placeholder convention:
-- :name means a value supplied by the application or by psql variables.

-- (1) Create a new user account.
INSERT INTO users(email, username, nickname, password_hash)
VALUES (:email, :username, :nickname, :password_hash)
RETURNING user_id, email, username, nickname, created_at;

-- (2) Create a new public channel inside a workspace by an authorized user.
-- The creator must already be a member of the workspace.
BEGIN;
WITH created_channel AS (
    INSERT INTO channels(workspace_id, name, channel_type, created_by)
    SELECT :workspace_id, :channel_name, 'public', :creator_id
    WHERE EXISTS (
        SELECT 1
        FROM workspace_members
        WHERE workspace_id = :workspace_id
        AND user_id = :creator_id
    )
)

```

```

    )
    RETURNING channel_id
)
INSERT INTO channel_members(channel_id, user_id)
SELECT channel_id, :creator_id
FROM created_channel
RETURNING channel_id, user_id;
COMMIT;

-- (3) For each workspace, list all current administrators.
SELECT w.workspace_id, w.name AS workspace, u.user_id, u.username, u.email
FROM workspaces AS w
JOIN workspace_members AS wm ON wm.workspace_id = w.workspace_id
JOIN users AS u ON u.user_id = wm.user_id
WHERE wm.role = 'admin'
ORDER BY w.name, u.username;

-- (4) For each public channel in a given workspace, count users invited
-- more than 5 days ago who have not yet joined.
SELECT c.channel_id,
       c.name AS channel,
       COUNT(ci.invited_user_id) AS pending_invites_older_than_5_days
FROM channels AS c
LEFT JOIN channel_invitations AS ci
  ON ci.channel_id = c.channel_id
 AND ci.status = 'pending'
 AND ci.invited_at < (:as_of::timestampz - INTERVAL '5 days')
 AND NOT EXISTS (
   SELECT 1
   FROM channel_members AS cm
   WHERE cm.channel_id = c.channel_id
        AND cm.user_id = ci.invited_user_id
 )
WHERE c.workspace_id = :workspace_id
     AND c.channel_type = 'public'
GROUP BY c.channel_id, c.name
ORDER BY c.name;

-- (5) For a particular channel, list all messages in chronological order.
SELECT m.message_id, m.posted_at, u.username AS sender, m.body
FROM messages AS m
JOIN users AS u ON u.user_id = m.sender_id
WHERE m.channel_id = :channel_id
ORDER BY m.posted_at, m.message_id;

-- (6) For a particular user, list all messages they have posted in any channel.
SELECT m.message_id, w.name AS workspace, c.name AS channel, m.posted_at, m.body
FROM messages AS m
JOIN channels AS c ON c.channel_id = m.channel_id
JOIN workspaces AS w ON w.workspace_id = c.workspace_id
WHERE m.sender_id = :user_id
ORDER BY m.posted_at, m.message_id;

-- (7) For a particular user, list all accessible messages containing a keyword.
-- Accessible means that the user is a member of both the workspace and the channel.
SELECT m.message_id,
       w.name AS workspace,
       c.name AS channel,
       u.username AS sender,

```

```

        m.posted_at,
        m.body
FROM messages AS m
JOIN channels AS c ON c.channel_id = m.channel_id
JOIN workspaces AS w ON w.workspace_id = c.workspace_id
JOIN users AS u ON u.user_id = m.sender_id
JOIN workspace_members AS wm
    ON wm.workspace_id = w.workspace_id
    AND wm.user_id = :user_id
JOIN channel_members AS cm
    ON cm.channel_id = c.channel_id
    AND cm.user_id = :user_id
WHERE m.body ILIKE '%' || :keyword || '%'
ORDER BY m.posted_at, m.message_id;

```

D Test Script

```

\set ON_ERROR_STOP on
\i schema.sql
\i sample_data.sql

\echo 'Task 1: create a new user account'
INSERT INTO users(email, username, nickname, password_hash)
VALUES ('frank@example.com', 'frank', 'Frank', 'hash_frank')
RETURNING user_id, email, username, nickname;

\echo 'Task 2: create a public channel by authorized workspace member Cara'
BEGIN;
WITH created_channel AS (
    INSERT INTO channels(workspace_id, name, channel_type, created_by)
    SELECT 1, 'announcements', 'public', 3
    WHERE EXISTS (
        SELECT 1 FROM workspace_members
        WHERE workspace_id = 1 AND user_id = 3
    )
    RETURNING channel_id, workspace_id, name, channel_type, created_by
),
added_member AS (
    INSERT INTO channel_members(channel_id, user_id)
    SELECT channel_id, 3 FROM created_channel
    RETURNING channel_id, user_id
)
SELECT created_channel.channel_id,
       created_channel.workspace_id,
       created_channel.name,
       created_channel.channel_type,
       created_channel.created_by,
       added_member.user_id AS initial_member
FROM created_channel
JOIN added_member USING (channel_id);
COMMIT;

\echo 'Task 3: current administrators per workspace'
SELECT w.name AS workspace, u.username, u.email
FROM workspaces AS w
JOIN workspace_members AS wm ON wm.workspace_id = w.workspace_id

```

```

JOIN users AS u ON u.user_id = wm.user_id
WHERE wm.role = 'admin'
ORDER BY w.name, u.username;

\echo 'Task 4: old pending invitations to public channels in Acme Lab as of 2026-04-26'
SELECT c.name AS channel,
       COUNT(ci.invited_user_id) AS pending_invites_older_than_5_days
FROM channels AS c
LEFT JOIN channel_invitations AS ci
  ON ci.channel_id = c.channel_id
 AND ci.status = 'pending'
 AND ci.invited_at < (TIMESTAMPTZ '2026-04-26 00:00:00-04' - INTERVAL '5 days')
 AND NOT EXISTS (
   SELECT 1 FROM channel_members AS cm
   WHERE cm.channel_id = c.channel_id
        AND cm.user_id = ci.invited_user_id
 )
WHERE c.workspace_id = 1
     AND c.channel_type = 'public'
GROUP BY c.channel_id, c.name
ORDER BY c.name;

\echo 'Task 5: all messages in #general'
SELECT m.message_id, to_char(m.posted_at, 'YYYY-MM-DD HH24:MI') AS posted_at, u.username, m.body
FROM messages AS m
JOIN users AS u ON u.user_id = m.sender_id
WHERE m.channel_id = 1
ORDER BY m.posted_at, m.message_id;

\echo 'Task 6: all messages posted by Bob'
SELECT m.message_id, w.name AS workspace, c.name AS channel, to_char(m.posted_at, 'YYYY-MM-DD HH24:MI') AS posted_at, m.body
FROM messages AS m
JOIN channels AS c ON c.channel_id = m.channel_id
JOIN workspaces AS w ON w.workspace_id = c.workspace_id
WHERE m.sender_id = 2
ORDER BY m.posted_at, m.message_id;

\echo 'Task 7: messages accessible to Cara containing perpendicular'
SELECT m.message_id, w.name AS workspace, c.name AS channel, u.username AS sender,
       to_char(m.posted_at, 'YYYY-MM-DD HH24:MI') AS posted_at, m.body
FROM messages AS m
JOIN channels AS c ON c.channel_id = m.channel_id
JOIN workspaces AS w ON w.workspace_id = c.workspace_id
JOIN users AS u ON u.user_id = m.sender_id
JOIN workspace_members AS wm
  ON wm.workspace_id = w.workspace_id
 AND wm.user_id = 3
JOIN channel_members AS cm
  ON cm.channel_id = c.channel_id
 AND cm.user_id = 3
WHERE m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at, m.message_id;

```

E Expected Test Results

Expected PostgreSQL test transcript summary
=====

Task 1: create a new user account

user_id	email	username	nickname
13	frank@example.com	frank	Frank

Task 2: create a public channel by authorized workspace member Cara

channel_id	workspace_id	name	channel_type	created_by	initial_member
13	1	announcements	public	3	3

Task 3: current administrators per workspace

workspace	username	email
Acme Lab	alice	alice@example.com
Acme Lab	bob	bob@example.com
Coop Board	cara	cara@example.com
Design Guild	hank	hank@example.com
Design Guild	maya	maya@example.com
Study Hub	ivy	ivy@example.com

Task 4: old pending invitations to public channels in Acme Lab as of 2026-04-26

channel	pending_invites_older_than_5_days
announcements	0
general	1
product	1
sprint-planning	1

Task 5: all messages in #general

message_id	posted_at	username	body
1	2026-04-02 08:20	alice	Welcome to Acme Lab general.
2	2026-04-03 10:00	bob	The perpendicular test rig is ready.
3	2026-04-03 12:30	cara	I can review the onboarding checklist.

Task 6: all messages posted by Bob

message_id	workspace	channel	posted_at	body
2	Acme Lab	general	2026-04-03 10:00	The perpendicular test rig is ready.
4	Acme Lab	product	2026-04-05 10:10	Product roadmap review at 3pm.

Task 7: messages accessible to Cara containing perpendicular

message_id	workspace	channel	sender	posted_at	body
2	Acme Lab	general	bob	2026-04-03 10:00	The perpendicular test rig is ready.
8	Coop Board	maintenance	alice	2026-04-04 12:15	The stair railing is perpendicular to the landing.

Access-control observation:

Cara is a member of Acme Lab and #general, and she is also a member of Coop Board and #maintenance. Therefore she sees Bob's public #general message and Alice's #maintenance message containing "perpendicular." She does not see Alice's private #hiring message containing the same keyword because she is not a member of that channel, and she also does not see unrelated matching messages from workspaces or channels where she lacks membership. This verifies that workspace membership alone is not enough; channel membership is also required.